

EVALUACIÓN PRÁCTICA FINAL DE UNIDAD

Escuela Profesional: Ingeniería de Sistemas

Ciclo: IX

Curso: Seguridad Informática

Unidad: IV — Monitoreo de Seguridad, SIEM e Inteligencia Artificial

Modalidad: Laboratorio / Evaluación Práctica

Duración: 4 horas

Puntaje Total: 20 puntos

Competencia evaluada: El estudiante diseña e implementa sistemas de monitoreo de seguridad utilizando herramientas SIEM y técnicas de análisis de eventos en tiempo real, integrando inteligencia artificial para la detección de anomalías y respondiendo proactivamente a incidentes en entornos empresariales.

INSTRUCCIONES GENERALES

- Trabaje de forma **individual**.
- **El estudiante debe crear su propio repositorio Git** con el nombre examen-practico-<apellido> (en minúsculas), inicializarlo localmente (`git init`) y publicarlo en GitHub/GitLab antes de iniciar el desarrollo. Todos los entregables deben estar commiteados y pusheados antes de que finalice el tiempo.
- **El estudiante es responsable de armar su entorno de trabajo:** instale y configure las herramientas necesarias (Python 3.11+, librerías pip, Wazuh, Elastic Stack 8.x, Jupyter Notebook) en su máquina virtual Linux (Ubuntu 22.04 LTS). Se evaluará que el entorno esté correctamente configurado y funcional.
- Documente el proceso de configuración del entorno en el archivo `README.md` del repositorio (versiones instaladas, comandos usados, capturas de servicios activos).
- **Se evaluará tanto el código escrito como la configuración de herramientas y servicios:** scripts funcionales, reglas activas en Wazuh, dashboards de monitoreo y evidencias de ejecución real tienen igual peso en la calificación.
- **Entrega de evidencias:** cada laboratorio exige screenshots específicos (ver sección *Guía de Evidencias y Screenshots*). Las capturas deben ser nítidas, mostrar fecha/hora del sistema y el nombre del estudiante visible en la barra de título o prompt de la terminal.

MODALIDAD AWS EDUCATION (Opción para recursos limitados)

Si el estudiante **no cuenta con los recursos computacionales suficientes** para ejecutar localmente las herramientas requeridas (mínimo recomendado: 8 GB RAM, 4 vCPU, 50 GB disco), podrá usar **AWS Academy / AWS Educate** como entorno de trabajo. Esta opción debe justificarse en el `README.md` indicando las especificaciones de su equipo personal.

Importante: el uso de AWS no exime de ningún entregable ni reduce los criterios de evaluación. Todos los laboratorios deben completarse igualmente.

Laboratorio	Servicio AWS sugerido	Notas
Lab 1 — Python/Logs	EC2 t3.micro (Ubuntu 22.04)	Subir datasets vía scp o S3
Lab 2 — Wazuh	EC2 t3.medium (mín. 2 GB RAM)	Instalar Wazuh All-in-One
Lab 3 — ML/Jupyter	Amazon SageMaker Studio Lab (gratuito) o EC2 t3.medium	Subir network_traffic.csv al entorno
Lab 4 — Dashboard	EC2 t3.small con Grafana OSS, o Amazon OpenSearch + Dashboards	Abrir puerto 3000/5601 en Security Group

Pasos mínimos para AWS: 1. Acceder a <https://aws.amazon.com/education/awseducate/> con el correo institucional. 2. Lanzar las instancias EC2 necesarias en la región us-east-1. 3. Incluir en el README.md: ID de instancia(s), región, AMI usada y comandos de instalación. 4. Los screenshots de evidencia deben mostrar el prompt con el hostname de la instancia EC2 (ej. ubuntu@ip-10-0-1-5) o la URL del servicio AWS.

LABORATORIO 1: Análisis Forense de Logs con Python (5 puntos)

Contexto

Se le proporcionan dos archivos de logs de un servidor web Apache y del servicio SSH de un servidor Linux. Descárguelos del repositorio del curso y cópielos a su carpeta lab1/:

Archivo	Descripción	Ruta en el repositorio del curso
auth.log	Log de autenticación SSH (500 líneas)	examenfinal/lab1/auth.log
access.log	Log de acceso Apache HTTP (1 000 líneas)	examenfinal/lab1/access.log

Una vez copiados, sus scripts deben leerlos desde la ruta relativa lab1/auth.log y lab1/access.log dentro de su propio repositorio.

Tarea 1.1 — Parseo y estadísticas de auth.log (1.5 pts)

Cree el script analizar_ssh.py que realice lo siguiente:

1. Lea el archivo auth.log e identifique todas las líneas con intentos de autenticación **fallidos** (Failed password).
2. Extraiga y cuente los intentos por **dirección IP de origen**.

3. Identifique las **10 IPs con mayor número de intentos fallidos** e imprima un ranking ordenado.
4. Genere una alerta en consola si alguna IP supera **50 intentos fallidos** con el mensaje:
[ALERTA] IP: <ip> – <n> intentos fallidos – Posible ataque de fuerza bruta
5. Exporte el resultado a un archivo `reporte_ssh.json` con la estructura:

```
{
  "fecha_analisis": "YYYY-MM-DD HH:MM:SS",
  "total_intentos_fallidos": 0,
  "ips_sospechosas": [
    { "ip": "x.x.x.x", "intentos": 0, "alerta": true }
  ]
}
```

Tarea 1.2 — Análisis de access.log (1.5 pts)

Cree el script `analizar_web.py` que realice lo siguiente:

1. Parsee el archivo `access.log` en formato Combined Log Format de Apache.
 2. Detecte patrones de **escaneo de directorios** (más de 20 peticiones a rutas distintas en menos de 60 segundos desde la misma IP).
 3. Identifique peticiones con códigos de respuesta **4xx y 5xx**, agrupadas por IP.
 4. Detecte posibles intentos de **SQL Injection** buscando en la URL los patrones: UNION, SELECT, --, OR 1=1, '.
 5. Genere el archivo `reporte_web.json` con los hallazgos.
-

Tarea 1.3 — Visualización (2 pts)

Usando `matplotlib` o `seaborn`, cree el script `visualizar.py` que genere las siguientes gráficas guardadas como imágenes `.png`:

1. **Gráfico de barras** — Top 10 IPs con más intentos fallidos SSH.
 2. **Línea de tiempo** — Número de peticiones HTTP por hora durante el día analizado.
 3. **Mapa de calor (heatmap)** — Peticiones HTTP por hora y código de respuesta (200, 301, 404, 500).
-

LABORATORIO 2: Reglas de Correlación en Wazuh (4 puntos)

Contexto

La empresa cuenta con Wazuh instalado y recibiendo logs del servidor SSH y del firewall. Su tarea es crear reglas de correlación personalizadas.

El directorio de trabajo es `/var/ossec/etc/rules/`.

Tarea 2.1 — Regla: Brute Force SSH (1.5 pts)

Cree el archivo `local_rules_ssh.xml` con una regla que:

1. Detecte **10 o más fallos de autenticación SSH** desde la misma IP en un intervalo de **60 segundos** (use `frequency` y `timeframe`).
2. Asigne nivel de severidad **10** y grupo `authentication_failures, brute_force`.
3. Genere la descripción: "Ataque de fuerza bruta SSH detectado desde `<ip_origen>`".

Ejemplo de estructura esperada:

```
<group name="local,ssh,">
  <rule id="100001" level="10" frequency="10" timeframe="60">
    <if_matched_sid>5716</if_matched_sid>
    <same_source_ip />
    <description>Ataque de fuerza bruta SSH desde $(srcip)</description>
    <group>authentication_failures,brute_force</group>
  </rule>
</group>
```

Tarea 2.2 — Regla: Exfiltración de datos (1.5 pts)

Cree el archivo `local_rules_exfil.xml` con una regla compuesta que:

1. Se dispare cuando se detecte **transferencia de datos saliente superior a 500 MB** en una sola conexión desde un host interno.
 2. Correlacione con un **login exitoso previo fuera del horario laboral** (entre 22:00 y 06:00).
 3. Nivel de severidad **14** (crítico) y descripción apropiada.
 4. Incluya comentario XML explicando la lógica de la regla.
-

Tarea 2.3 — Prueba y evidencia (1 pt)

1. Reinicie el servicio Wazuh Manager: `systemctl restart wazuh-manager`.
 2. Simule un ataque de fuerza bruta con el script provisto `simular_bruteforce.sh`.
 3. Capture **screenshot o copia del log** `/var/ossec/logs/alerts/alerts.log` mostrando que la regla disparó correctamente.
 4. Incluya la evidencia en la carpeta `lab2/evidencia/` del repositorio.
-

LABORATORIO 3: Modelo de Detección de Anomalías con ML (6 puntos)

Contexto

Se le proporciona el dataset `network_traffic.csv` con 10 000 registros de tráfico de red capturados durante 30 días. Descárguelo del repositorio del curso:

- **Ruta en el repositorio del curso:** `examenfinal/lab3/network_traffic.csv`

Cópielo a la carpeta `lab3/` de su repositorio. El notebook debe leerlo desde la ruta relativa `lab3/network_traffic.csv`. Las columnas son:

Columna	Descripción
<code>timestamp</code>	Fecha y hora del evento
<code>src_ip</code>	IP de origen
<code>dst_ip</code>	IP de destino
<code>dst_port</code>	Puerto de destino
<code>protocol</code>	Protocolo (TCP/UDP/ICMP)
<code>bytes_sent</code>	Bytes enviados
<code>bytes_recv</code>	Bytes recibidos
<code>duration_sec</code>	Duración de la conexión en segundos
<code>packets</code>	Número de paquetes
<code>label</code>	<code>normal</code> / <code>anomaly</code> (solo para validación — NO usar en entrenamiento)

Tarea 3.1 — Exploración y preprocesamiento (1.5 pts)

En el Jupyter Notebook `deteccion_anomalias.ipynb`:

1. Cargue el dataset con `pandas` y muestre estadísticas descriptivas.
2. Visualice la distribución de `bytes_sent` y `duration_sec` con histogramas.
3. Identifique y trate valores nulos o atípicos extremos.
4. Realice **feature engineering**: cree al menos 2 nuevas variables derivadas (ej. `ratio_bytes`, `bytes_por_segundo`).
5. Normalice las features numéricas con `StandardScaler`.

Tarea 3.2 — Entrenamiento del modelo (2 pts)

En el mismo notebook:

1. Entrene un modelo de **Isolation Forest** con los datos (excluyendo la columna `label`).
 - Parámetros sugeridos: `contamination=0.05`, `n_estimators=100`, `random_state=42`.
2. Obtenga las predicciones (`-1` = anomalía, `1` = normal).
3. Calcule las métricas de evaluación usando la columna `label`:
 - **Precision, Recall, F1-Score** y **matriz de confusión**.
4. Grafique la matriz de confusión con `seaborn`.

Tarea 3.3 — Interpretación y umbral dinámico (1.5 pts)

1. Grafique el **score de anomalía** (`decision_function`) para todos los registros.
 2. Identifique manualmente el **umbral óptimo** que maximice el F1-Score usando una curva de umbral vs F1.
 3. Liste los **Top 10 registros más anómalos** del dataset con todas sus columnas y explique en markdown dentro del notebook por qué podrían representar una amenaza real.
-

Tarea 3.4 — Exportación del modelo (1 pt)

1. Serialice el modelo entrenado con `joblib: modelo_anomalias.pkl`.
2. Cree el script `predecir.py` que:
 - Cargue `modelo_anomalias.pkl`.
 - Acepte como argumento un archivo CSV nuevo.
 - Imprima en consola los registros clasificados como anomalía con sus scores.

Uso esperado:

```
python predecir.py nuevo_trafico.csv
```

LABORATORIO 4: Dashboard de Monitoreo (5 puntos)

Contexto

El estudiante puede elegir **cualquiera** de las siguientes herramientas para construir el dashboard de monitoreo SOC. Justifique brevemente en el `README.md` la herramienta elegida e incluya los pasos de instalación/configuración.

Opción	Herramienta	Fuente de datos sugerida
A	Kibana (Elastic Stack 8.x)	Índice <code>wazuh-alerts-*</code> en Elasticsearch
B	Grafana (v10+)	Loki para logs o Elasticsearch como datasource
C	OpenSearch Dashboards	Índice Wazuh en OpenSearch

Independientemente de la herramienta elegida, las tareas y los criterios de evaluación son equivalentes. Los ejemplos de configuración que se muestran a continuación usan Kibana como referencia.

Tarea 4.1 — Conexión a la fuente de datos y exploración (1 pt)

1. Configure la fuente de datos en la herramienta elegida (p. ej. en Kibana: index pattern wazuh-alerts-*; en Grafana: datasource Elasticsearch/Loki apuntando a los logs de Wazuh).
 2. Explore los eventos de las últimas 24 horas y exporte 20 registros representativos a CSV como evidencia.
-

Tarea 4.2 — Visualizaciones (2 pts)

Cree las siguientes **4 visualizaciones** en la herramienta elegida:

#	Tipo	Métrica	Agrupación
V1	Vertical Bar	Count de alertas	Por nivel de severidad (Wazuh rule.level)
V2	Data Table	Top 10 IPs con más alertas	data.srcip
V3	Line	Alertas por hora (últimas 24h)	@timestamp por intervalo 1h
V4	Pie Chart	Distribución por tipo de regla	rule.groups

Tarea 4.3 — Dashboard integrado (1.5 pts)

1. Cree un **Dashboard** llamado "SOC – Monitor de Seguridad" que integre las 4 visualizaciones anteriores.
 2. Configure un **filtro/rango de tiempo global** de las últimas 24 horas.
 3. Añada un panel de texto (Markdown en Kibana / Text panel en Grafana) con el título del dashboard y los datos del autor.
 4. Exporte el dashboard en el formato nativo de la herramienta (dashboard_soc.json en Kibana/Grafana, o el equivalente) e inclúyalo en el repositorio.
-

Tarea 4.4 — Alerta de umbral (0.5 pts)

Configure una **alerta de umbral** en la herramienta elegida:

- **Kibana:** cree una Alert Rule de tipo threshold que se active cuando alertas de nivel ≥ 10 superen **5 eventos en 5 minutos**; notifique al conector Index (soc-notificaciones).
- **Grafana:** cree un Alert Rule en el panel V3 (línea de tiempo) que se dispare si el conteo supera **5 en 5 minutos**; notifique al canal de alerta configurado (webhook, email o similar).
- **Otra herramienta:** configure una alerta equivalente con criterio y umbral similares.

Capture screenshot de la regla configurada como evidencia.

ENTREGABLES Y ESTRUCTURA DEL REPOSITORIO

El estudiante debe **crear y publicar** el repositorio Git (`git init` → primer commit → `git remote add origin <url>` → `git push`) al inicio del examen y mantenerlo actualizado con commits frecuentes. El repositorio debe incluir un archivo `README.md` que documente: versiones de herramientas instaladas, comandos de configuración del entorno y pasos para reproducir cada laboratorio.

La estructura final debe ser:

```
examen-practico-<apellido>/
├── README.md                                ← Documentación general
├── lab1/
│   ├── analizar_ssh.py
│   ├── analizar_web.py
│   ├── visualizar.py
│   ├── reporte_ssh.json                    ← Generado al ejecutar
│   ├── reporte_web.json                  ← Generado al ejecutar
│   ├── graficas/                          ← Imágenes .png generadas
│   │   ├── top10_ssh.png
│   │   ├── timeline_http.png
│   │   └── heatmap_http.png
│   └── evidencias/                        ← Screenshots requeridos Lab 1
│       ├── SCR-1.1a_ssh_ejecucion.png
│       ├── SCR-1.1b_ssh_json.png
│       ├── SCR-1.2a_web_ejecucion.png
│       └── SCR-1.2b_web_json.png
├── lab2/
│   ├── local_rules_ssh.xml
│   ├── local_rules_exfil.xml
│   └── evidencias/                        ← Screenshots requeridos Lab 2
│       ├── SCR-2.1_wazuh_activo.png
│       ├── SCR-2.2_reglas_validadas.png
│       └── SCR-2.3_alerta_disparada.png
├── lab3/
│   ├── deteccion_anomalias.ipynb
│   ├── predecir.py
│   ├── modelo_anomalias.pkl
│   └── evidencias/                        ← Screenshots requeridos Lab 3
│       ├── SCR-3.1_eda.png
│       ├── SCR-3.2_metricas.png
│       ├── SCR-3.3_umbral_f1.png
│       └── SCR-3.4_predecir.png
└── lab4/
    ├── dashboard_soc.json                ← Export del dashboard (Kibana/Grafana/otro)
    ├── datasource_config.json            ← Config de fuente de datos exportada (opcional)
    └── evidencias/
        ├── herramienta_usada.txt        ← Nombre, versión y URL/IP del servicio
        └── SCR-4.1_fuente_datos.png
```


- SCR-4.2_visualizaciones.png
- SCR-4.3_dashboard.png
- SCR-4.4_alerta.png

Convención de nombres para screenshots: usar el prefijo SCR-<lab>.<area>_<descripcion>.png en minúsculas y sin espacios. Ejemplo: SCR-1.1_ejecucion_analizar_ssh.png.

GUÍA DE EVIDENCIAS Y SCREENSHOTS REQUERIDOS

A continuación se detalla **qué debe mostrarse en cada screenshot** y **dónde guardarlo** en el repositorio. Un screenshot que no cumpla los requisitos de contenido no será contabilizado.

Lab 1 — Análisis Forense de Logs

Código	Archivo a guardar	Qué debe mostrar el screenshot
SCR-1.1a	lab1/evidencias/SCR-1.1a_ssh_ejecucion.png	Terminal ejecutando python analizar_ssh.py con las líneas [ALERTA] visibles en consola
SCR-1.1b	lab1/evidencias/SCR-1.1b_ssh_json.png	Contenido del archivo reporte_ssh.json (terminal con cat o editor)
SCR-1.2a	lab1/evidencias/SCR-1.2a_web_ejecucion.png	Terminal ejecutando python analizar_web.py mostrando detecciones de escaneo y SQLi
SCR-1.2b	lab1/evidencias/SCR-1.2b_web_json.png	Contenido del archivo reporte_web.json
SCR-1.3*	lab1/graficas/top10_ssh.png, timeline_http.png, heatmap_http.png	Las 3 gráficas generadas (se entregan directamente como archivos PNG)

Lab 2 — Reglas Wazuh

Código	Archivo a guardar	Qué debe mostrar el screenshot
SCR-2.1	lab2/evidencias/SCR-2.1_wazuh_activo.png	Salida de systemctl status wazuh-manager con estado active (running)

Código	Archivo a guardar	Qué debe mostrar el screenshot
SCR-2.2	lab2/evidencias/SCR-2.2_reglas_validadas.png	Validación XML de las reglas sin errores (ej. xmllint --noout local_rules_ssh.xml && echo OK)
SCR-2.3	lab2/evidencias/SCR-2.3_alerta_disparada.png	Extracto de /var/ossec/logs/alerts/alerts.log o consola Wazuh mostrando la alerta de brute-force con la IP atacante y el rule.id configurado

Lab 3 — Modelo ML

Código	Archivo a guardar	Qué debe mostrar el screenshot
SCR-3.1	lab3/evidencias/SCR-3.1_eda.png	Jupyter Notebook con las celdas de EDA ejecutadas e histogramas visibles
SCR-3.2	lab3/evidencias/SCR-3.2_metricas.png	Celda con Precision, Recall, F1-Score impresos y gráfica de la matriz de confusión
SCR-3.3	lab3/evidencias/SCR-3.3_umbral_f1.png	Gráfica curva umbral vs F1 y tabla Top 10 anomalías en el notebook
SCR-3.4	lab3/evidencias/SCR-3.4_predecir.png	Terminal ejecutando python predecir.py nuevo_trafico.csv con registros anómalos en la salida

Lab 4 — Dashboard

Código	Archivo a guardar	Qué debe mostrar el screenshot
SCR-4.1	lab4/evidencias/SCR-4.1_fuente_datos.png	Interfaz de la herramienta con la fuente de datos conectada y al menos 1 evento visible
SCR-4.2	lab4/evidencias/SCR-4.2_visualizaciones.png	Las 4 visualizaciones creadas (puede ser collage o screenshot de cada una individualmente como SCR-4.2a, SCR-4.2b, etc.)

Código	Archivo a guardar	Qué debe mostrar el screenshot
SCR-4.3	lab4/evidencias/SCR-4.3_dashboard.png	Dashboard completo con nombre "SOC – Monitor de Seguridad" y datos del autor visibles
SCR-4.4	lab4/evidencias/SCR-4.4_alerta.png	Regla de alerta configurada con umbral, condición y acción de notificación visibles

Para entornos AWS: todos los screenshots deben mostrar en la barra de título del navegador o en el prompt de la terminal el hostname/IP de la instancia EC2 o la URL del servicio AWS usado, como prueba de que se ejecutó en la nube.

RÚBRICA DE EVALUACIÓN

Laboratorio 1 — Análisis Forense de Logs (5 pts)

Criterio	Excelente (100 %)	Parcial (60 %)	Insuficiente (0 %)
1.1 Parseo SSH y JSON	Script funciona, JSON correcto, alertas generadas; código limpio y bien estructurado	Script funciona parcialmente o código con errores menores	No ejecuta o no genera JSON
1.2 Análisis web y detección SQLi	Detecta escaneo, SQLi y genera JSON; lógica de detección correcta en el código	Solo detecta uno de los dos o lógica incompleta	No detecta ni genera reporte
1.3 Visualizaciones	Las 3 gráficas correctas, guardadas y con títulos/etiquetas	1–2 gráficas generadas	Sin gráficas

Laboratorio 2 — Reglas Wazuh (4 pts)

Criterio	Excelente (100 %)	Parcial (60 %)	Insuficiente (0 %)
2.1 Regla brute force	XML válido con sintaxis Wazuh correcta, regla dispara y aparece en alertas	XML con errores menores de sintaxis o configuración	XML inválido o no dispara
2.2 Regla exfiltración	Lógica completa, nivel correcto, comentarios en el XML explicando cada elemento	Regla parcial o sin comentarios	Ausente

Criterio	Excelente (100 %)	Parcial (60 %)	Insuficiente (0 %)
2.3 Evidencia + configuración	Servicio Wazuh activo, screenshot con alerta real en logs, README documenta los pasos de configuración	Screenshot incompleto o sin documentación	Sin evidencia ni configuración demostrable

Laboratorio 3 — Modelo ML (6 pts)

Criterio	Excelente (100 %)	Parcial (60 %)	Insuficiente (0 %)
3.1 Exploración y preprocesamiento	EDA completo, features derivadas, normalización	EDA parcial	Sin preprocesamiento
3.2 Modelo y métricas	Isolation Forest funcional, $F1 > 0.7$, conf. matrix	Modelo entrena pero métricas incompletas	No entrena
3.3 Interpretación y umbral	Curva umbral-F1, Top 10 con explicación	Parcial	Sin análisis
3.4 Exportación	modelo.pkl y predecir.py funcionales	Solo uno de los dos	Ausente

Laboratorio 4 — Dashboard de Monitoreo (5 pts)

Criterio	Excelente (100 %)	Parcial (60 %)
4.1 Fuente de datos y exploración	Herramienta instalada y configurada, fuente de datos conectada, CSV de 20 registros exportado	Herramienta instalada pero fuente de datos con errores o sin CSV
4.2 Visualizaciones	4 visualizaciones correctas con títulos y configuración apropiada en la herramienta elegida	2–3 visualizaciones
4.3 Dashboard integrado	Dashboard completo con filtro de tiempo, panel de texto del autor y export JSON en el repositorio	Dashboard sin filtro, sin panel de texto o sin export
4.4 Alerta	Regla de umbral configurada en la herramienta elegida, con screenshot de evidencia en el repositorio	Configurada sin evidencia o sin archivos en repositorio

ESCALA DE CALIFICACIÓN

Puntaje	Calificación	Nivel de logro
18 – 20	Excelente	Supera el logro esperado
14 – 17	Bueno	Logro esperado
11 – 13	Regular	En proceso
0 – 10	Deficiente	En inicio